

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Dockerfile & Image Best Practices

This is the canonical requirements and reference page for Dockerfile and image quality. Use [Creating Docker Environment](#) for starter Dockerfiles, compose shapes, and local build commands; use this page when you need to understand or fix Dockerfile CI checks.

All Terminal-Bench task images must be:

PROPERTY	DESCRIPTION
Reproducible	The same source builds to the same image over time and across environments
Cacheable	Common layers are shared across tasks
Lazy-pull friendly	Startup-critical files are accessible without pulling the full image
Auditable	Images are digest-pinned, signed, labeled, and free of any secrets
Complete	Tasks run without network access — images must contain all dependencies
Siloed	The image must not leak task solutions or tests
Resourced	Tasks must define CPU, memory, and storage needs in task definition

CI Enforcement Summary

Three Dockerfile checks block by default:

CHECK	REQUIREMENT
<code>check_pinned_images</code>	Every <code>FROM</code> image must be digest-pinned w
<code>check_sanctioned_base_images</code>	The final runtime base image must be sanction
<code>check_build_context_size</code>	<code>environment/</code> must be at most 100 MiB tot

The remaining Dockerfile checks warn by default, but warning checks can still emit structural errors when required files such as `environment/` or `environment/Dockerfile` are missing.

1. Pin Base Images by Digest

Every `FROM` line must use an immutable digest. Never use floating tags such as `latest`. Tags may be included for readability, but the digest is the source of truth. CI will fail any Dockerfile with a `FROM` line lacking `@sha256`. Update digests deliberately, in reviewable commits.

DOCKERFILE	Copy
<pre># Bad FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm # Good FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256</pre>	

2. Use Sanctioned Runtime Base Images

Prefer the **canonical Terminal-Bench base images** below over ad hoc public images for the final runtime stage. Builder stages may use task-appropriate

toolchain images, but the final stage is what agents and verifiers run in and is checked by CI.

Why a canonical list: Tasks across the dataset have been fragmenting onto many slightly different base images (different tags, different registries, different patch versions) for no real reason. Reusing a small set of canonical images dramatically improves cache hit rates, reduces verifier startup time, and keeps the dataset's runtime surface area manageable.

Canonical base images

Use the **exact digest-pinned reference** from this list whenever possible. The list intentionally collapses minor version variants into one canonical entry per family (e.g., the Go entry covers all Go 1.21–1.26 majors plus alpine/bullseye/bookworm variants — use the canonical row).

Language runtimes

CANONICAL IMAGE
public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256:01f42367a0...
public.ecr.aws/docker/library/node:22-bookworm-slim@sha256:f3a68cf41a855d...
public.ecr.aws/docker/library/golang:1.24-bookworm@sha256:1a6d4452c65dea3...
public.ecr.aws/docker/library/rust:1.85-slim@sha256:9f841bbe9e7d8e37ceb96...
public.ecr.aws/docker/library/eclipse-temurin:21-jdk-jammy@sha256:25d1276...
public.ecr.aws/docker/library/gcc:13-bookworm@sha256:930f2ebe239275fa6722...
public.ecr.aws/docker/library/ruby:3.3-slim-bookworm@sha256:e76733e94b3a5...



Build tools and SDKs

CANONICAL IMAGE

`public.ecr.aws/docker/library/maven:3.9.9-eclipse-temurin-21@sha256:3a4ab...`

Distro bases (for minimal or custom images)

CANONICAL IMAGE

`public.ecr.aws/docker/library/debian:bookworm-slim@sha256:4724b8cc51e33e3...`

`public.ecr.aws/docker/library/ubuntu:24.04@sha256:0d39fcc8335d6d74d5502f6...`

Using a non-canonical base image

Tasks **may** use a base image not on this list if there's a genuine reason — for example, a runtime the canonical list doesn't cover, a hardware-specific image, or a stack that requires a niche distro. The acceptance gate works as follows:

BASE IMAGE USED	JUSTIFICATION
Canonical (from the tables above)	Not required
Non-canonical	Present and credible (in <code>Dockerfile</code> comment)
Non-canonical	Empty / missing / boilerplate

Examples of acceptable justifications:

- "The canonical Java image is JDK-only; this task requires a full JRE-plus-system-libraries setup."

- "Targeting a new language not yet in the canonical list (e.g., Zig, Crystal)."
- "Stress-testing behavior specific to a Red Hat-derived distribution that Debian-derived canonical images don't reproduce."

A reviewer can reject a non-canonical base if the justification is missing, vague, or matches an existing canonical entry (i.e., the canonical image would have worked fine).

What a good base-image family provides

- Shell utilities required by the harness (`tmux` , `asciinema`)
- Observability tooling, if required
- Standard CA certificates and locale configuration
- Common verifier/runtime bootstrap tools
- A pinned package-manager baseline

3. Make Builds Reproducible

Pin all dependencies and avoid nondeterministic build steps.

Pin language dependencies with lockfiles:

LANGUAGE	LOCKFILE
Python	<code>requirements.txt</code> with exact versions, <code>uv.lock</code> , <code>poetry.lock</code>
Node	<code>package-lock.json</code> , <code>pnpm-lock.yaml</code> , <code>yarn.lock</code>
Rust	<code>Cargo.lock</code>
Go	<code>go.mod</code> and <code>go.sum</code>
JVM	Pinned Maven/Gradle dependency locks where possible

Pin downloaded binaries by version and checksum. Avoid `curl | sh` unless the artifact is version-pinned and checksum-verified:

DOCKERFILE Copy

```
# Bad
RUN curl -LsSf https://astral.sh/uv/install.sh | sh

# Good
ARG UV_VERSION=0.6.14
ARG UV_SHA256=...
RUN curl -fsSL "https://github.com/astral-sh/uv/releases/download/$
    && echo "${UV_SHA256} /tmp/uv.tar.gz" | sha256sum -c - \
    && tar -xzf /tmp/uv.tar.gz -C /usr/local/bin --strip-components
    && rm /tmp/uv.tar.gz
```

Also avoid build steps that embed:

- Wall-clock time
- Random paths or hostnames
- Usernames
- Nondeterministic generated output

Set deterministic build environment variables where applicable (e.g., `SOURCE_DATE_EPOCH`). Use BuildKit/buildx for consistent builds.

4. Structure Layers from Least Volatile to Most Volatile

Order Dockerfile operations so task edits only invalidate top layers.

Requirements:

- Put base OS and runtime dependencies before task-specific source
- Copy manifest files before copying source files
- Install dependencies before copying frequently edited files
- Do not use `COPY . /app` unless the task directory is minimal and `.dockerignore` is strict

- Avoid squashing all content into one layer
- Aim for a moderate number of purposeful layers

Preferred pattern:

DOCKERFILE

 Copy

```
FROM <base>@sha256:...

# 1. OS/system packages
RUN apt-get update \
    && apt-get install -y --no-install-recommends ... \
    && rm -rf /var/lib/apt/lists/*

# 2. Runtime/package-manager setup
COPY requirements.txt /tmp/requirements.txt
RUN pip install --no-cache-dir -r /tmp/requirements.txt

# 3. Project manifests
COPY pyproject.toml uv.lock /app/

# 4. Dependency fetch/install
RUN ...

# 5. Source and task files
COPY src/ /app/src/
COPY data/ /app/data/

# 6. Task-specific build step, if needed
RUN ...

WORKDIR /app
```

5. Consolidate apt Usage Correctly

Use one apt transaction per stage and volatility class.

Requirements:

- Do not run `apt-get upgrade`

- Do not leave `/var/lib/apt/lists/*` in the image
- Do not install recommended packages unless required
- Do not install build tools in the runtime stage unless the task requires them
- Prefer moving common apt packages into sanctioned base images

DOCKERFILE Copy

```
# Bad
RUN apt-get update && apt-get install -y git
RUN apt-get update && apt-get install -y tmux asciinema
RUN apt-get update && apt-get install -y python3 python3-pip

# Good
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        asciinema \
        ca-certificates \
        git \
        python3 \
        python3-pip \
        tmux \
    && rm -rf /var/lib/apt/lists/*
```

6. Keep Build Tools Out of Runtime Images

Use multi-stage builds whenever the Dockerfile compiles or bundles artifacts. If the task requires the agent to compile or build code, toolchains in the final image are fine.

Use multi-stage builds for: `cargo build`, `go build`, `mvn package`, `gradle build`, `npm ci && npm run build`, `dotnet publish`, C/C++ builds with `make` / `cmake`, or similar.

DOCKERFILE Copy

```
FROM public.ecr.aws/docker/library/rust:1.85-slim@sha256:... AS bui
WORKDIR /build
```

```

COPY Cargo.toml Cargo.lock ./
COPY src/ src/
RUN cargo build --release --locked

FROM public.ecr.aws/docker/library/debian:bookworm-slim@sha256:...
RUN apt-get update \
    && apt-get install -y --no-install-recommends ca-certificates \
    && rm -rf /var/lib/apt/lists/*
COPY --from=builder /build/target/release/my-tool /usr/local/bin/my
WORKDIR /app

```

7. Images Must Contain All Dependencies

All tasks must operate correctly without network access during the agent run and verifier step.

Requirements:

- `tmux` and `asciinema` **must** be installed — the agent runtime requires both to start a session. Missing either will cause all agent runs to fail with no verifier output.
- All package downloads happen at image build time
- `test.sh` must not use `curl`, `wget`, `pip install`, `npm install`, `cargo fetch`, `mvn dependency:get`, or similar networked operations
- Python wheels, npm packages, Maven artifacts, Cargo registry state, reference binaries, and fixtures must be preloaded during build
- `task.toml` must set `allow_internet = false`
- The Oracle agent must pass with network access disabled
- Agents must be able to complete the task without any missing assets or dependencies

8. Separate Agent-Visible Runtime from Verifier-Only Assets

Solution files, hidden tests, and privileged assets must never be accessible to the agent.

Requirements:

- Do not copy `solution/` into the runtime image
- Do not copy hidden tests into agent-visible locations
- Do not store expected outputs in writable agent paths
- Do not let the verifier derive ground truth from files the agent can modify
- Place reference binaries or fixtures in controlled verifier-only locations
- Public fixtures used by the task may be copied into the image; hidden verifier assets must stay isolated

9. Use `.dockerignore` and Narrow COPY

All non-trivial tasks must include a `.dockerignore` file.

Requirements:

- Keep `environment/` at or below 100 MiB total
- Keep each individual file in `environment/` at or below 50 MiB
- Prefer `COPY file file` or `COPY src/ src/` over `COPY . .`
- Do not copy local caches, virtualenvs, build outputs, `.git`, editor files, or credentials

Recommended `.dockerignore` template:

```
.git
.gitignore
**/__pycache__
**/*.pyc
**/.pytest_cache/
**/.mypy_cache/
**/.ruff_cache/
**/node_modules/
**/target/
**/dist/
**/build/
**/.venv/
```

 Copy

```
**/venv/  
.env  
*.log  
solution/  
tests/
```

10. Store Files as Files, Not Heredocs or Opaque Archives

Source code should be visible as source files on disk, not embedded in the Dockerfile.

DOCKERFILE

 Copy

```
# Bad  
RUN cat > /app/foo.py <<'EOF'  
print('hello world')  
EOF  
  
# Good  
COPY hello_world.py /app/hello_world.py
```

Extract archives at build time so snapshotters can lazy-load individual files:

DOCKERFILE

 Copy

```
# Bad  
COPY fixtures.tar.gz /tmp/fixtures.tar.gz  
  
# Good  
COPY fixtures.tar.gz /tmp/fixtures.tar.gz  
RUN mkdir -p /app/fixtures \  
    && tar -xzf /tmp/fixtures.tar.gz -C /app/fixtures \  
    && rm /tmp/fixtures.tar.gz
```

11. Use Metadata-Preserving COPY Options

Only change permissions for files that actually need it. Do not recursively rewrite metadata across `/app`.

DOCKERFILE Copy

```
# Bad
RUN chmod -R 755 /app
RUN chown -R appuser:appuser /app

# Good
COPY --chmod=0755 run.sh /usr/local/bin/run-task
COPY --chown=appuser:appuser src/ /app/src/
```

12. General Image Hygiene

Unless required by the task, images must not contain:

- `.git`, `.env`, credentials or tokens, SSH keys
- Package-manager or compiler caches
- Unused build tools or test fixtures
- Local editor files or vendor documentation unrelated to the task
- Large unused datasets or hidden solution files

Perform cleanup in the same layer that creates files:

DOCKERFILE Copy

```
# Bad
RUN apt-get update
RUN apt-get install -y build-essential
RUN rm -rf /var/lib/apt/lists/*

# Good
RUN apt-get update \
    && apt-get install -y --no-install-recommends build-essential \
    && rm -rf /var/lib/apt/lists/*
```

13. Optimise for Lazy Pull and Hot-Path Locality

Put startup-critical files in earlier, smaller layers. Put cold optional assets in later layers.

Startup-critical files for Terminal tasks typically include:

- Shell entrypoints
- Task instructions
- Small runtime scripts
- Dependency metadata
- Small fixtures needed by the first command

Requirements:

- Keep large optional datasets out of the image where possible; mount or fetch them at runtime when the task design and platform allow it
- Avoid compressing many runtime files into a single archive
- If using eStargz or Nydus, ensure the image converts cleanly and provide a prioritized file list for startup-critical paths

14. Declare Task Resources Explicitly

`task.toml` must include an `[environment]` section with realistic resource definitions. Use peak resources observed from real agent runs. The Oracle must always pass with the defined limits.

Requirements:

- CPU, memory, and storage must reflect actual task needs
- Do not over- or under-request resources
- Build timeout must account for compilation-heavy tasks but remain bounded
- Agent and verifier timeouts must be realistic

TOML

 Copy

```
[agent]
timeout_sec = 900

[verifier]
timeout_sec = 420

[environment]
build_timeout_sec = 600.0
docker_flags = []
cpus = 1
memory_mb = 2048
storage_mb = 10240
gpus = 0
gpu_types = []
allow_internet = false
```

15. Publish Images with Audit Metadata

Include OpenContainers labels to support downstream auditing:

DOCKERFILE

[Copy](#)

```
LABEL org.opencontainers.image.source="..."
LABEL org.opencontainers.image.revision="..."
LABEL org.opencontainers.image.created="..."
LABEL org.opencontainers.image.version="..."
LABEL org.opencontainers.image.licenses="..."
```

Next Steps

- [Creating Docker Environment](#) — Basic Dockerfile setup
- [Task Components](#) — Full task file structure
- [CI Checks Reference](#) — Automated checks your Dockerfile must pass

PREVIOUS

[Docker Environment Setup](#)

NEXT

[Writing Oracle Solution](#)